

Practice Questions on Feed Forward Neural Networks

1. A FF neural network parameters for implementing the function $y = x_1 \text{AND} x_2$ is possible with just one output unit followed by a ReLU activation connected directly to the inputs. Give the value of parameters W_1^o, W_2^o, b^o that achieves that. ..1 $W_1^o = 1, W_2^o = 1, b^o = -1$
2. A feed forward neural network can represent any function. Our goal is to design a neural network to multiply two integers $\mathbf{x}^1, \mathbf{x}^2$ expressed as k bit binary vectors $x_1^1, \dots, x_k^1$ and $x_1^2, \dots, x_k^2$ respectively. For example, if $k = 3, \mathbf{x}^1 = [1 \ 0 \ 1], \mathbf{x}^2 = [0 \ 1 \ 0]$, then the network should output 10. Show the layers and the optimal values of each parameter. Use ReLU as the activation function. ..4 At the first layer, we maintain k^2 units. The i, j -th unit computes $h_{i,j}^1 = \text{ReLU}(2^i 2^j + Mx_j^1 + Mx_i^2 - 2M)$ where M is greater than 2^{2k} . When $x_i^1 = x_j^2 = 1$ the value of $h_{i,j}^1 = 2^i 2^j$ Else it is zero. The next layer simply adds these values.
Some of you propose solutions that involve a multiplication of two hidden outputs or two inputs. Such answers are incorrect since you are only allowed linear transforms followed by ReLU in standard feed forward networks.
3. Consider a multi-layered feed forward neural network where the activation function at each hidden unit instead of ReLU is the sign function where $\text{sign}(h) = 1$ if $h \geq 0$ and 0 if $h < 0$. Design a method of representing decision trees of *arbitrary depth* using a neural network with a *constant* number of (e.g., two) hidden layers.

Assume binary class labels, all continuous attributes. Each internal node j of the decision tree is a binary condition of the form “attribute $x_j < \text{constant } c_j$ ” and has a left branch and a right branch depending on whether the condition is true or false. Each leaf node j of the tree has a label L_j that is either “+1” or “-1”. Assume the tree is balanced, that is, all leaves are at the same level.

For each unit of your neural network specify the values of the parameters of the linear layer connecting it to the previous layer units. Use W_{jk}^l to denote the weight connecting j th output of layer $l - 1$ to k th unit of layer l and use b_k^l to denote the bias parameter of node k at layer l . Use W_j^o to denote the weight connecting j th output of last layer to the output node and use b^o for output bias. (Hint: two hidden layers suffice) ..6

For each internal node j of the tree with condition of the form $x_j < c_j$, create a unit n in the first hidden layer connecting j to the attribute x_j with $W_{nj}^1 = -1$ and $b_n^1 = c_j$ present in its split condition with weight 1. The sign activation on this node will output a 1 when the condition $x_j < c_j$ is true and false otherwise. Now add a second layer corresponding to each of the leaf node of the tree. For each leaf node n , let $(n_1, s_1), \dots, (n_k, s_k)$ denote the nodes along the path from the root to the leaf with s_r denoting whether the condition at node n_r is true or false. Connect unit of leaf node j to nodes $n_1, \dots, n_k$ of previous layer. The weight $W_{jn_r}^2$ is +1 if s_r is true else it is -1. The bias b_j^2 is $-(k_j - 0.5)$ where k_j is the number of true (left) branches in the path to this node. This implies that only when all k_j conditions are true the bias term is activated. Now add a last output node that is connected with weight “ L_j ” to the leaf units with label “ L_j ”. The bias of the output node is 0.

4. For the XOR network discussed in class, workout the forward and backward computation step for training at some initial value of the parameters and when the input is both ones.
5. Let W_{jk}^l denote the weight connecting j th output of layer $l - 1$ to k th unit of layer l in a 2 hidden-layer feed-forward network with a last linear layer and a least square loss. The

activation unit at each hidden layer is a ReLU. For an example $(\mathbf{x}, y)$, let h_k^l denote the k th output of the l -th layer o denote the output from the network. Write the expression for the gradient of loss on this example wrt W_{12}^1 . Let the number of hidden units be m in each of the two hidden layers. Your answer should be in the full expanded form and not in recursive form like in Backpropagation.

$$-(y - o) \sum_{j=1}^m W_{j1}^3 \delta(h_j^2 > 0) W_{2j}^2 \delta(h_2^1 > 0) x_1$$

6. State two ill-effects of initializing parameters of a feed forward neural network with all zeros during training.

(a) Reason 1: ..2

The symmetries of the hidden layers are not broken. So, all hidden units might take the same parameter value.

(b) Reason 2: ..2

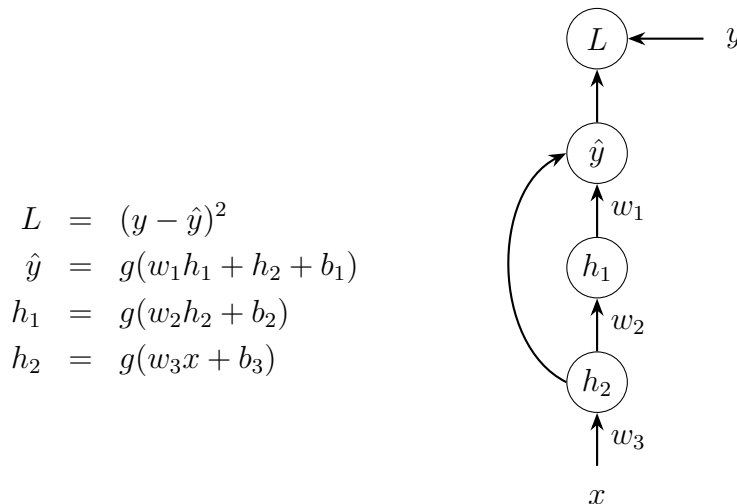
The ReLU will be saturated and gradient information will not flow below the ReLU layer.

7. Answer the following questions about the training of a neural network parameters using the backpropagation algorithm.

(a) State two different conditions under which a feed-forward network in a particular stochastic gradient update step may result in no change to the network parameters. In other words, under what conditions would the gradient of the loss with respect to all parameters be zero. Justify your answer. 1. If the network is trained using a convex loss, and we find an $\hat{y}$ for which the loss is minimized. In this case, the gradient of the loss will be zero.

2. If all ReLUs are saturated. ..3

(b) Consider this feed-forward neural network variant with a residual connection to the last layer as follows:



Assume, $g()$ is the ReLU function. If initial values of all w_j parameters is -1 and b_j parameters is 0, $x = -2, y = 1$, write the gradient of the loss with respect to the w_3 parameter.

..2

- (c) Based on the above example, why do you think residual connections help during neural network training? They help us update parameters of lower layer even if the activation of higher layers is saturated. ..2

8. Exercise 13.1 from the PML textbook.